Machine Learning: k-Means and PCA for Regime Detection

Zhidian Jing, Siqi Zhang, Stefaan Fernando, Daniel Greenstein, Oscar Peyron
University College London – COMP0040

April 2026

1. Introduction

2. Data Processing and Feature Engineering

This project focuses on regime detection on multi-dimensional financial time-series. We track 9 major stock indices across 3 different regions (America, Europe, Asia) to develop our analysis.

Table 1: Index universe by region.

Region	Index	Ticker
America	S&P 500	`^GSPC`
America	Dow Jones	`^DJI`
America	Nasdaq	`^IXIC`
Europe	FTSE 100	`^FTSE`
Europe	CAC 40	`^FCHI`
Europe	DAX	`^GDAXI`
Asia	KOSPI	`^KS11`
Asia	Nikkei 225	`^N225`
Asia	Hang Seng	`^HSI`

We test our algorithm on 3 different regions each time using the 3 indices from that region, so that we can compare the results across different markets and time periods.

2.1. Raw Data Structure

Each index is downloaded from Yahoo Finance with daily OHLCV columns: `Date`, `Open`, `High`, `Low`, `Close`, and `Volume`. After merging, the full DataFrame uses a MultiIndex on columns of the form `(index_name, OHLCV_field)`, so that, for example, S&P 500 closing prices are accessed via `df["SP500"]["Close"]`.

2.2. Feature Engineering

From the closing prices two families of features are computed.

Log-returns. Over multiple horizons (default: 1, 5, and 21 trading days):

$$r_{t,p} = \ln\left(\frac{P_t}{P_{t-p}}\right). \tag{1}$$

Rolling annualised volatility. Default windows of 21 and 63 days:

$$\sigma_{t,w} = \mathrm{std}(r_{t-w:t}) \times \sqrt{252}\,. \tag{2}$$

This produces a feature matrix of shape $(T,\ 9 \times (3 \text{ returns} + 2 \text{ volatilities})) = (T, 45)$ columns, where each row corresponds to one trading day and each column to a pair `(index_name, feature_name)`.

Rolling Correlation. In order to capture the dynamic nature of regime, features are computed in a rolling window fashion, so that the feature vector at time t is based on data from the previous w days. For example, the rolling correlation between two indices X and Y at time t with window w is computed as:

$$\rho_{X,Y,t} = \frac{\sum_{i=0}^{w-1}(r_{X,t-i} - \bar{r}_X)(r_{Y,t-i} - \bar{r}_Y)}{\sqrt{\sum_{i=0}^{w-1}(r_{X,t-i} - \bar{r}_X)^2 \ \sum_{i=0}^{w-1}(r_{Y,t-i} - \bar{r}_Y)^2}} \tag{3}$$

where $\bar{r}_X$ and $\bar{r}_Y$ are the mean returns of indices X and Y over the window w.

Z-score We additionally compute momentum Z-score features to measure mean-reversion by normalising the distance between the current price and the moving average by the rolling volatility:

$$Z_{t,w} = \frac{P_t - \mathrm{MA}_{t,w}}{\sigma_{t,w}} \tag{4}$$

RSI Finally, we compute the Relative Strength Index (RSI) to capture overbought and oversold conditions. Mathematically, this translates to the following:

$$\mathrm{RSI}_t = 100 - \frac{100}{1 + \frac{\text{Average Gain}_t}{\text{Average Loss}_t}} \tag{5}$$

where Average Gain_t and Average Loss_t are computed over a specified period (commonly 14 days) as the average of positive and negative price changes, respectively.

Building feature set We then merge for all the indices the features (log-returns lagged , rolling volatility, correlation, z-score, RSI) into a single DataFrame. We compute the 1 day, 5 day, and 21 day log-returns for each index, as well as the rolling volatility with windows of 21 and 63 days which gives us 15 features in total. Moreover, we compute the rolling pearson-correlation on a 63-day window for each pair of indices, resulting in 3 correlation features. Finally, we compute the momentum z-score on 63-day window and the RSI with a 14-day period for each index, adding 6 more features. Hence, the final raw feature matrix, has 24 columns with the number of rows associated to the number of trading days minus the warm-up period for the rolling features.

The number of rows depend on the considered region. For instance, for the American indices the period from 2006 to 2020, account for 3523 trading days. The warm-up period accounts for 63 days, since th biggest rolling window is 63 days, which means that the final feature matrix has exactly 3460 rows and 24 columns. We then scale the features using standardisation and apply PCA to reduce dimensionality before feeding the data into the clustering algorithm. The number of principal components is selected to retain 95% of the variance, which results in a reduced feature space of 7 dimensions for American Indices, 10 dimensions for European Indices, and 14 dimensions for Asian Indices. The following tables summarise the feature engineering process and the resulting feature set dimensions for each region after PCA.

Table 2: Feature engineering summary.

Region	Feature Set Dimensions
America	(3460, 7) (after PCA)
Europe	(3523, 10) (after PCA)
Asia	(3561, 14) (after PCA)

2.3. Pipeline Summary

Yahoo Finance API

↓

Raw OHLCV (per index)

↓

Merged DataFrame (MultiIndex columns)

↓

Close prices only (simple DataFrame)

↓

Log-returns (1d, 5d, 21d) Feature Engineerin

↓

Feature matrix → Clustering / PCA

Figure 1: Data pipeline from raw download to feature matrix.

3. Mathematical Foundations

In this project we apply two main machine learning techniques for regime detection: k-means clustering and Principal Component Analysis (PCA). We then compare the results of these techniques against a more traditional approach to regime detection, the Hidden Markov Model with Gaussian Mixtures (GMMHMM). This section therefore provides the mathematical foundations for these three techniques, starting with k-means and PCA, and then moving on to GMMHMM.

3.1. K-Means Clustering

K-means clustering is an unsupervised learning algorithm that partitions the data into K clusters by minimising the within-cluster sum of squares. The algorithm iteratively assigns each data point to the nearest cluster centroid and then updates the centroids based on the assigned points.

The Within-Cluster Sum of Squares (WCSS) is defined as:

$$\text{WCSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \tag{6}$$

where C_k is the set of points in cluster k and μ_k is the centroid of cluster k.

However, when dealing with higher-dimensional data, the standard k-means algorithm can be limited due to the curse of dimensionality as well as the different topologies of the clusters. To address this, one can reduce dimensionality using PCA before applying k-means.

3.2. Principal Component Analysis (PCA)

We use PCA to reduce the dimensionality of our feature space. In our case we reduce the 24 features from the 3 indices within a region, into a smaller set of principal components that capture the majority of the variance in the data. PCA works by finding the eigenvectors of the covariance matrix $\mathbf{\Sigma}$ of the data, which represent the directions of maximum variance. The eigenvalues associated with these eigenvectors indicate the amount of variance captured by each principal component. Mathematically, we express this in the following way:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \tag{7}$$

where $\mathbf{X}$ is the centered data matrix, $\mathbf{U}$ contains the left singular vectors, $\mathbf{\Sigma}$ is a diagonal matrix of singular values, and $\mathbf{V}$ contains the right singular vectors (principal components). We select the top d^* components that retain a specified percentage of the variance (e.g., 95%).

4. Hidden Markov Models with Gaussian Mixtures (GMMHMM)

Hidden Markov Models are used in sequence prediciton where states are hidden and cannot be directly observed. It models two stochastic processses: a hidden Markov chain of states, and a directly visible set of observations dependent on those states.

The Markovian property asserts that the current state depends only on the previous state, not the entire history. For states $S_1, S_2, ..., S_t$:

$$\mathbb{P}(S_t = a | S_1, S_2, ..., S_{t-1}) = \mathbb{P}(S_t = a | S_{t-1}) \tag{8}$$

The model is specified by a set of features and a number of regimes. It returns a set of transition probabilites, $A_{i,j}$, where $\sum_j A_{i,j} = 1$ and

$$A_{i,j} = \mathbb{P}(S_t = j | S_{t-1} = i) \tag{9}$$

and i, j are regime states.

In addition, the model computes an emission distribtion for observed states conditional on the

hidden state. Formally, for observed state X_t, we say that $X_t|S_t = i \sim \mathcal{N}(\mu_i, \Sigma_i)$ A standard Hidden Markov model uses a single Gaussian distrubtion for each hidden state. However, markets are often heavy tailed and skewed, and a single gaussian distbrution may not accurately capture these properties. We therefore use a mixture of Gaussian distrutions, the combination of which more accurately models the extreme events in markets. With a gaussian mixture

$$X_t \mid S_t = i \sim \sum_{k}^{m} w_{i,k} \mathcal{N}(\mu_{i,k}, \Sigma_{i,k}) \tag{10}$$

where m represents the number of mixtures, $w_{i,k}$ is the mixture weight such that $\sum_k w_{i,k} = 1$, and $\mu_{i,k}, \Sigma_{i,k}$ represent the mixture mean and covariance.

Procedure:

1. The GMMHMM randomly initalises a transition matrix A, mean and covariances for the Gaussians and mixture weights for each state distribution where hidden states $S_t \in \{1, \ldots, K\}$
2. The model then peforms a Baum–Welch (EM) algorithm
3. In the expectation step, the algorithm computes the probabilty of hidden states and mixture components using the observed data
4. In the maximisation step, the model parameters are reselected after log likelihood maximisation
5. The algorithm is repeatedly run until desired refinement is reached - the number of cyles is a hyperparameter

For this work, the hidden states are the market regimes. The number of regimes is selected via minimising BIC. The number of mixtures is a hyperparameter.

5. Proposed Algorithm

In this section, we present the complete pipeline for regime detection using k-means clustering and PCA. The algorithm consists of the following stages:

Algorithm 1 K-Means Regime Detection Pipeline

Require: Closing price matrix $\mathbf{P} \in \mathbb{R}^{T \times N}$ for N indices over T trading days
Require: PCA variance threshold $\alpha \in (0, 1]$, candidate range $[K_{\min}, K_{\max}]$
Ensure: Regime labels $\ell_t \in \{0, \ldots, K^* - 1\}$ for each day t, ordered by mean return

— **Stage 1: Feature Engineering** —
for each index $n = 1, \ldots, N$ **do**
  for each horizon $p \in \{1, 5, 21\}$ **do**
    $r_{t,p}^{(n)} \leftarrow \ln(P_t^{(n)} / P_{t-p}^{(n)})$ ▷ Log-returns
  end for
  for each window $w \in \{21, 63\}$ **do**
    $\sigma_{t,w}^{(n)} \leftarrow \mathrm{std}(r_{t\text{-}w:t}^{(n)}) \times \sqrt{252}$ ▷ Rolling volatility
  end for
  $Z_t^{(n)} \leftarrow (P_t^{(n)} - \mathrm{MA}_{t,63}^{(n)}) / \sigma_{t,63}^{(n)}$ ▷ Momentum z-score
  $\mathrm{RSI}_t^{(n)} \leftarrow \mathrm{RSI}_{14}(P^{(n)})$ ▷ Relative Strength Index
end for
for each pair (i, j) with $i < j$ **do**
  $\rho_t^{(i,j)} \leftarrow \mathrm{RollingCorr}_{63}(r^{(i)}, r^{(j)})$ ▷ Pairwise correlation
end for
Concatenate all features into $\mathbf{F} \in \mathbb{R}^{T' \times D}$ and drop warm-up rows

— **Stage 2: Standardisation** —
for each feature $d = 1, \ldots, D$ **do**
  $\tilde{F}_{t,d} \leftarrow (F_{t,d} - \hat{\mu}_d) / \hat{\sigma}_d$ ▷ Z-score normalisation
end for

— **Stage 3: Dimensionality Reduction** —
$\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}^\top \leftarrow \mathrm{SVD}(\tilde{\mathbf{F}})$
Select $d^* = \min\{d : \sum_{i=1}^{d} \sigma_i^2 / \sum_{i=1}^{D} \sigma_i^2 \geq \alpha\}$
$\mathbf{X} \leftarrow \tilde{\mathbf{F}}\, \mathbf{V}_{:,1:d^*} \in \mathbb{R}^{T' \times d^*}$ ▷ Project onto top d^* PCs

— **Stage 4: Optimal K Selection** —
for $K = K_{\min}, \ldots, K_{\max}$ **do**
  Run K-Means on $\mathbf{X}$ with K clusters
  Record inertia $\mathcal{I}(K)$ and silhouette score $\mathcal{S}(K)$
end for
$K^* \leftarrow \arg\max_K \ \mathcal{S}(K)$ ▷ Best silhouette

— **Stage 5: Final Clustering** —
$\hat{\ell}_1, \ldots, \hat{\ell}_{T'} \leftarrow \mathrm{KMeans}(\mathbf{X},\ K^*)$

— **Stage 6: Regime Ordering** —
for each cluster $k = 0, \ldots, K^* - 1$ **do**
  $\bar{r}_k \leftarrow \mathrm{mean}(\{r_t : \hat{\ell}_t = k\})$ ▷ Mean basket return
end for
$\pi \leftarrow \mathrm{argsort}(\bar{r}_0, \ldots, \bar{r}_{K^*-1})$ ▷ Rank worst → best
$\ell_t \leftarrow \pi^{-1}(\hat{\ell}_t) \quad \forall\, t$ ▷ Relabel: 0 = bearish, $K^* - 1$ = bullish

return $\boldsymbol{\ell} = (\ell_1, \ldots, \ell_{T'})$

6. Results

In order to evaluate the performance of our proposed k-means and PCA pipeline for regime detection, we compare the results against a more traditional approach using Hidden Markov Models with Gaussian Mixtures (GMMHMM). We also compared the results with a peak-to-trough method, which is a simple rule-based approach that identifies market regimes based on the peaks and troughs of the index values. We parametrized this peak-to-trough method by setting the maximum drawdown threshold to 20%, which means that a new regime is identified whenever the index experiences a drawdown of 20% or more from its previous peak. Mathematically, we define the drawdown at time t as:

$$\text{Drawdown}_t = \frac{P_t - P_{\text{peak}}}{P_{\text{peak}}} \tag{11}$$

where P_{peak} is the maximum index value observed up to time t. A new regime is triggered when $\text{Drawdown}_t \leq -0.20$. We then compare the different regime labelling methods, by creating non-realistic trading strategies, that go long when the label indicates bullish regime and flat when the label indicates bearish regime, and we compare the resulting cumulative returns of these strategies across the different labelling methods. For american indices we observe the following :

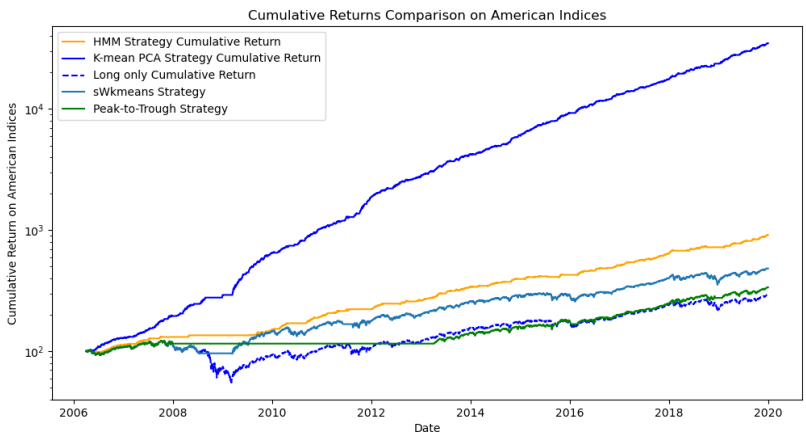

Figure 2: Cumulative returns of trading strategies based on different regime detection methods for American indices.

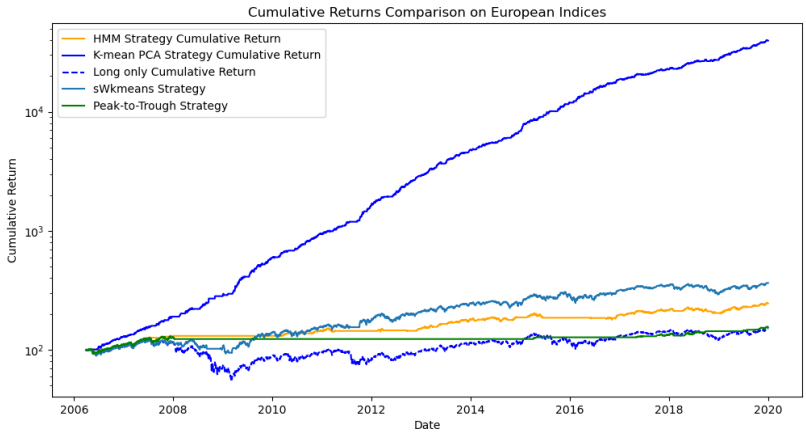

Figure 3: Cumulative returns of trading strategies based on different regime detection methods for European indices.

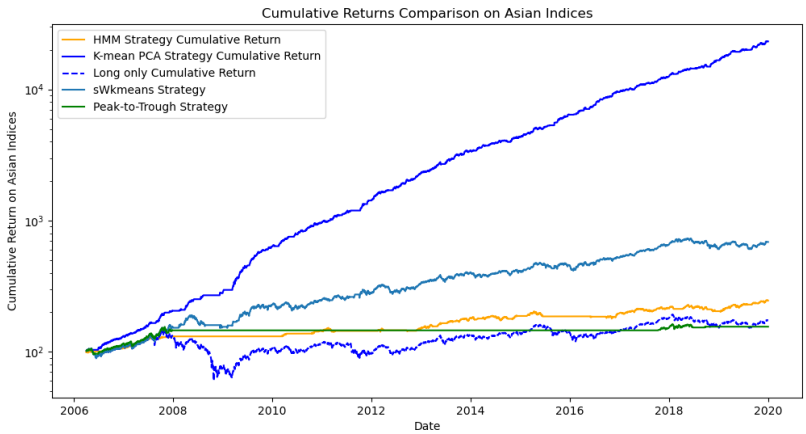

Figure 4: Cumulative returns of trading strategies based on different regime detection methods for Asian indices.

We obse

7. Conclusion